Machine Dynamics

Quick Return Mechanism

Kinematics Dynamics and Cam Design

Pouya Asghari

Instructor: Dr. Rastgoo

School of Mechanical Engineering
College of Engineering, University of Tehran
Second semester of academic year 1403–1404
Submission date: 1404/03/16 (Iranian calendar)

About this edition

This is an English translation of the supplied Persian report. The analysis, reported dimensions, source equations, comparison figures, and code are retained. Mathematical notation is transcribed from the source rather than presented as independently verified equations. The student number is omitted.

The source acknowledges a discrepancy in member 5's angular velocity. The original code and native models have not been executed during translation. A separate editorial review identifies further code and reproducibility checks. Native files are preserved unchanged in the repository package; the largest MATLAB recording remains in the original archive.

Contents

1 Introduction **2**

2 Kinematic Analysis **2**
2.1 Initial quick-return design . . . 3
2.2 Vector loop technique . . . 4
2.3 Vector equations . . . 5

3 Dynamic Analysis **8**
3.1 Dynamic equations . . . 8

4 Cam Design **11**
4.1 Cam construction . . . 12

5 Simulation and Comparison **14**
5.1 Adams model . . . 14
5.2 Comparison plots . . . 14
5.3 SolidWorks design . . . 20

6 Conclusion **22**

7 References **22**

A MATLAB Source Listing **22**

B Original Appendix Output Figures **30**

C Editorial Review and Reproducibility **34**

Abstract

This project focuses on the kinematic and dynamic analysis of a quick-return mechanism that converts uniform rotation into nonuniform reciprocating linear motion. A computational model is developed to determine member positions, velocities, accelerations, joint forces, and input torque. A MATLAB program simulates two motion cycles and plots angular velocities and accelerations. The dynamic analysis extends the calculation to joint forces and input torque over a nominal ten-second interval, presented as plots and a force matrix.

In the second stage, the driving crank, member 2, is replaced by a cam arrangement intended to reproduce the follower's angular displacement. A cam profile is developed and compared with the target displacement using commercial mechanism simulation. The complete mechanism is animated and modeled for comparison. The project combines analytical modeling and software simulation to investigate motion reproduction and dynamic behavior. Statements of validation in the original report refer to these software comparisons, not to physical experiments.

Keywords: quick-return mechanism; kinematic analysis; dynamic analysis; MATLAB; joint forces; Adams/View; cam design.

1 Introduction

Mechanism analysis and design are important when a machine must generate a particular motion efficiently. Quick-return mechanisms produce reciprocating motion with different forward and return speeds and are used in machine tools and mechanical transmission systems. This project examines their kinematics and dynamics and designs a cam to replace one of the original members.

The principal aim is to investigate the mechanism's motion and develop a model for predicting member velocities, accelerations, and joint forces. First, analytical kinematics are implemented in MATLAB and compared with Adams/View. Next, the dynamic analysis determines joint forces and input-crank torque. Finally, a driving member is replaced with a cam and the resulting motion is compared with the target displacement.

The report presents the analysis, design, simulation, results, and discussion step by step.

2 Kinematic Analysis

The link lengths and initial angles are selected to permit continuous complete motion. Angular and linear velocities and accelerations are then calculated and compared with commercial simulation.

2.1 Initial quick-return design

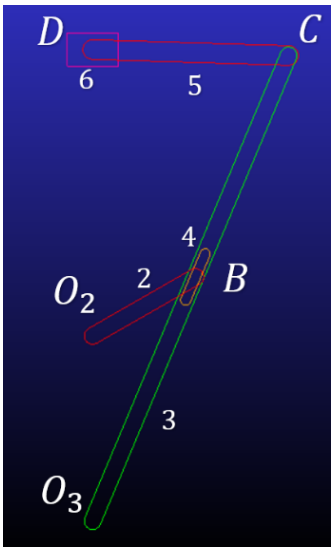


Figure 1: Initial quick-return mechanism in Adams.

Trial and error in Adams gives the following joint coordinates and member dimensions. Values are in meters unless otherwise indicated.

$$O_3\,(0,0)$$

$$O_2(0,0.11)$$

$$B\,(0.061,0.145)$$

$$C\,(0.1163,0.2765)$$

$$D(0,0.28)$$

$$C\bar{O}_3 = 0.3\angle 67.184^\circ$$

$$B\bar{O}_2 = 0.07\angle 30^\circ$$

$$D\bar{C} = 0.1164$$

All regular links have a width of 0.01 m and depth of 0.008 m. Gravity is enabled in the simulation because it is also included in the analytical calculations.

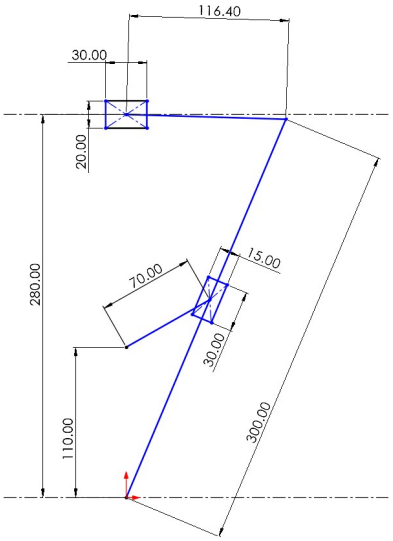


Figure 2: Member dimensions; drawing dimensions are in millimeters.

Member 2 is pinned to slider 4, which slides along member 3. Slider 4 must be wider than member 3; its external width is chosen as 15 mm and depth as 12 mm.

2.2 Vector loop technique

Two vector loops are defined for position, velocity, and acceleration analysis. The lower loop contains the fixed ground vector, the input crank vector, and the variable vector along the slotted member. The ground vector is known, and the crank length and input angle are prescribed, allowing the remaining vector length and angle to be found. Differentiating once and twice gives angular velocity, slider velocity, angular acceleration, and slider acceleration; the crank angular acceleration is zero. The upper loop includes the crank, the remaining portion of member 3, connecting member 5, a fixed vertical ground segment, and a variable horizontal displacement. The complementary member-3 segment is related to the lower-loop vector by reversing direction and subtracting its length from the total member length. The unknowns are member 5's angle and the output displacement. They are solved in the same manner. The source reuses several ground-vector symbols; the diagram and supplied code clarify their roles.

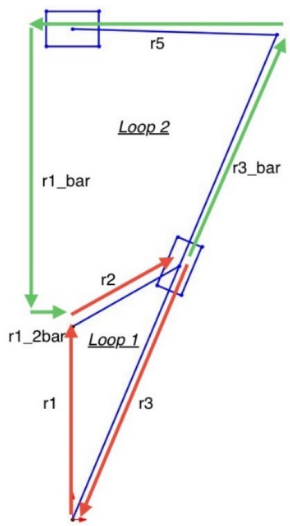


Figure 3: Vector loops used in the analytical model.

2.3 Vector equations

The general approach is implemented using the following complex-vector loop equations. Original mathematical notation is retained; ambiguities and potential errors require checking against the code.

Loop I

$$r_1 + r_2 + r_3 = 0 \rightarrow r_1 e^{i\theta_1} + r_2 e^{i\theta_2} + r_3 e^{i\theta_3} = 0 \rightarrow r_3, \theta_3 = determined$$

$$\frac{d}{dt} \rightarrow ir_1\dot{\theta}_1 e^{i\theta_1} + ir_2\dot{\theta}_2 e^{i\theta_2} + ir_3\dot{\theta}_3 e^{i\theta_3} + \dot{r}_3 e^{i\theta_3} = 0 \rightarrow \dot{\theta}_3, \dot{r}_3 = determined$$

$$\frac{d}{dt} \rightarrow i\dot{r}_2\dot{\theta}_2 e^{i\theta_2} + ir_2\left(\ddot{\theta}_2 + \dot{\theta}_2^2\right) e^{i\theta_2} + 2i\dot{r}_3\dot{\theta}_3 e^{i\theta_3} + ir_3\left(\ddot{\theta}_3 + \dot{\theta}_3^2\right) e^{i\theta_3} + \ddot{r}_3 e^{i\theta_3} = 0$$

$$\rightarrow \ddot{\theta}_3, \ddot{r}_3 = determined$$

Loop II

$$\bar{r}_1 + \bar{r}_1 + r_2 + \bar{r}_3 + r_5 = 0 \rightarrow \bar{r}_1 e^{i\bar{\theta}_1} + \bar{r}_1 e^{i\bar{\theta}_1} + r_2 e^{i\theta_2} + \bar{r}_3 e^{i\bar{\theta}_3} + r_5 e^{i\theta_5} = 0$$

$$\rightarrow \bar{r}_1, \theta_5 = determined$$

$$\frac{d}{dt} \rightarrow i\bar{r}_1\dot{\bar{\theta}}_1 e^{i\bar{\theta}_1} + \dot{\bar{r}}_1 + ir_2\dot{\theta}_2 e^{i\theta_2} + \dot{\bar{r}}_3 e^{i\bar{\theta}_3} + i\bar{r}_3\dot{\theta}_3 e^{i\bar{\theta}_3} + ir_5\dot{\theta}_5 e^{i\theta_5} = 0$$

$$\rightarrow \dot{\bar{r}}_1, \dot{\theta}_5 = determined$$

$$\frac{d}{dt} \rightarrow \ddot{\bar{r}}_1 + ir_2\left(i\dot{\theta}_2^2 e^{i\theta_2}\right) + \ddot{\bar{r}}_3 e^{i\bar{\theta}_3} + 2i\dot{\bar{r}}_3\dot{\theta}_3 e^{i\bar{\theta}_3} + i\bar{r}_3(\ddot{\theta}_3 + \dot{\theta}_3^2)e^{i\bar{\theta}_3} + ir_5(\ddot{\theta}_5 + \dot{\theta}_5^2)e^{i\theta_5} = 0 \rightarrow \ddot{\bar{r}}_1, \ddot{\theta}_5 = determined$$

Each complex equation can be separated into real and imaginary equations and solved. This determines the mechanism's kinematic quantities.

The equations are solved in MATLAB. The source reports that vpasolve requires approximately 5–10 minutes, whereas a Newton–Raphson implementation using a Jacobian reduces the runtime to below ten seconds. These are reported timings, not independently repeated benchmarks. Newton–Raphson requires initial conditions. The MATLAB input angle begins at zero and spans two revolutions, whereas the previously illustrated geometry begins at 30 degrees. SolidWorks is therefore used to determine the dimensions and angles for a zero-degree initial crank position.

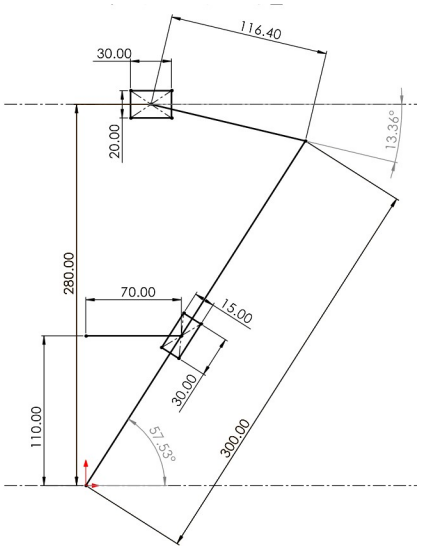


Figure 4: Geometry for the zero-degree initial crank angle.

The initial conditions are introduced into the solution procedure. The known and unknown quantities are arranged as follows.

Loop I

$$\begin{cases} r_1 cos\theta_1 + r_2 cos\theta_2 + r_3 cos\theta_3 = 0 \\ r_1 sin\theta_1 + r_2 sin\theta_2 + r_3 sin\theta_3 = 0 \end{cases}$$

$$\frac{d}{dt} \rightarrow \begin{cases} -r_2\dot{\theta}_2 sin\theta_2 + \dot{r}_3 cos\theta_3 - r_3\dot{\theta}_3 sin\theta_3 = 0 \\ r_2\dot{\theta}_2 cos\theta_2 + \dot{r}_3 sin\theta_3 + r_3\dot{\theta}_3 cos\theta_3 = 0 \end{cases}$$

$$\rightarrow \begin{cases} -r_2 sin\theta_2 + \dot{r}'_3 cos\theta_3 - r_3\dot{\theta}'_3 sin\theta_3 = 0 \\ r_2 cos\theta_2 + \dot{r}'_3 sin\theta_3 + r_3\dot{\theta}'_3 cos\theta_3 = 0 \end{cases}$$

$$\rightarrow \begin{bmatrix} cos\theta_3 & -r_3 sin\theta_3 \\ sin\theta_3 & r_3 cos\theta_3 \end{bmatrix} \begin{bmatrix} \dot{r}'_3 \\ \dot{\theta}'_3 \end{bmatrix} = \begin{bmatrix} r_2 sin\theta_2 \\ -r_2 cos\theta_2 \end{bmatrix}$$

$$\frac{d}{dt} \rightarrow \begin{cases} -r_2\dot{\theta}_2^2 cos\theta_2 + \ddot{r}_3 cos\theta_3 - 2\dot{r}_3\dot{\theta}_3 sin\theta_3 - r_3\left(\ddot{\theta}_3 sin\theta_3 + \dot{\theta}_3^2 cos\theta_3\right) = 0 \\ -r_2\dot{\theta}_2^2 sin\theta_2 + \ddot{r}_3 sin\theta_3 + 2\dot{r}_3\dot{\theta}_3 cos\theta_3 + r_3\left(\ddot{\theta}_3 cos\theta_3 - \dot{\theta}_3^2 sin\theta_3\right) = 0 \end{cases}$$

$$\rightarrow \begin{cases} -r_2 cos\theta_2 + \ddot{r}'_3 cos\theta_3 - 2\dot{r}_3\dot{\theta}_3 sin\theta_3 - r_3\left(\ddot{\theta}'_3 sin\theta_3 + \dot{\theta}_3^2 cos\theta_3\right) = 0 \\ -r_2 sin\theta_2 + \ddot{r}'_3 sin\theta_3 + 2\dot{r}_3\dot{\theta}_3 cos\theta_3 + r_3\left(\ddot{\theta}'_3 cos\theta_3 - \dot{\theta}_3^2 sin\theta_3\right) = 0 \end{cases}$$

$$\rightarrow \begin{bmatrix} cos\theta_3 & -r_3 sin\theta_3 \\ sin\theta_3 & r_3 cos\theta_3 \end{bmatrix} \begin{bmatrix} \ddot{r}'_3 \\ \ddot{\theta}'_3 \end{bmatrix} = \begin{bmatrix} r_2 cos\theta_2 + 2\dot{r}_3\dot{\theta}_3 sin\theta_3 + r_3\dot{\theta}_3^2 cos\theta_3 \\ r_2 sin\theta_2 - 2\dot{r}_3\dot{\theta}_3 cos\theta_3 + r_3\dot{\theta}_3^2 sin\theta_3 \end{bmatrix}$$

Loop II

$$\begin{cases} r_2 cos\theta_2 + \bar{r}_3 cos\theta_3 + r_5 cos\theta_5 + \bar{r}_1 cos\bar{\theta}_1 + \bar{r}_1 cos\bar{\theta}_1 = 0 \\ r_2 sin\theta_2 + \bar{r}_3 sin\theta_3 + r_5 sin\theta_5 + \bar{r}_1 sin\bar{\theta}_1 + \bar{r}_1 sin\bar{\theta}_1 = 0 \end{cases}$$

$$\frac{d}{dt} \rightarrow \begin{cases} r_2\dot{\theta}_2 sin\theta_2 + \dot{\bar{r}}_3 cos\theta_3 + \bar{r}_3 sin\bar{\theta}_3 + r_5\dot{\theta}_5 sin\theta_5 + \dot{\bar{r}}_1 cos\bar{\theta}_1 = 0 \\ r_2\dot{\theta}_2 cos\theta_2 + \dot{\bar{r}}_3 sin\theta_3 + \bar{r}_3 cos\bar{\theta}_3 + r_5\dot{\theta}_5 cos\theta_5 + \dot{\bar{r}}_1 sin\bar{\theta}_1 = 0 \end{cases}$$

$$\rightarrow \begin{cases} r_2 sin\theta_2 + \dot{\bar{r}}_3 cos\theta_3 + \bar{r}_3 sin\bar{\theta}_3 + r_5\dot{\theta}'_5 sin\theta_5 + \dot{\bar{r}}'_1 cos\bar{\theta}_1 = 0 \\ r_2 cos\theta_2 + \dot{\bar{r}}_3 sin\theta_3 + \bar{r}_3 cos\bar{\theta}_3 + r_5\dot{\theta}'_5 cos\theta_5 + \dot{\bar{r}}'_1 sin\bar{\theta}_1 = 0 \end{cases}$$

$$\rightarrow \begin{bmatrix} -r_5 sin\theta_5 & cos\bar{\theta}_1 \\ r_5 cos\theta_5 & sin\bar{\theta}_1 \end{bmatrix} \begin{bmatrix} \dot{\theta}'_5 \\ \dot{\bar{r}}'_1 \end{bmatrix} = \begin{bmatrix} r_2 sin\theta_2 - \dot{\bar{r}}_3 cos\bar{\theta}_3 + \bar{r}_3\dot{\bar{\theta}}_3 sin\bar{\theta}_3 \\ r_2 cos\theta_2 - \dot{\bar{r}}_3 sin\bar{\theta}_3 - \bar{r}_3\dot{\bar{\theta}}_3 cos\bar{\theta}_3 \end{bmatrix}$$

$$\frac{d}{dt} \rightarrow \begin{cases} -r_2\dot{\theta}_2^2 cos\theta_2 + \ddot{\bar{r}}_3 cos\bar{\theta}_3 - 2\dot{\bar{r}}_3\dot{\bar{\theta}}_3 sin\bar{\theta}_3 - \bar{r}_3\left(\ddot{\bar{\theta}}_3 sin\bar{\theta}_3 + \dot{\bar{\theta}}_3^2 cos\bar{\theta}_3\right) \\ -r_5\left(\ddot{\theta}_5 sin\theta_5 + \dot{\theta}_5^2 cos\theta_5\right) + \ddot{\bar{r}}_1 cos\bar{\theta}_1 = 0 \\ -r_2\dot{\theta}_2^2 sin\theta_2 + \ddot{\bar{r}}_3 sin\bar{\theta}_3 + 2\dot{\bar{r}}_3\dot{\bar{\theta}}_3 cos\bar{\theta}_3 + \bar{r}_3\left(\ddot{\bar{\theta}}_3 cos\bar{\theta}_3 + \dot{\bar{\theta}}_3^2 sin\bar{\theta}_3\right) \\ +r_5\left(\ddot{\theta}_5 cos\theta_5 - \dot{\theta}_5^2 sin\theta_5\right) + \ddot{\bar{r}}_1 sin\bar{\theta}_1 = 0 \end{cases}$$

$$\rightarrow \begin{cases} -r_2 cos\theta_2 + \ddot{\bar{r}}_3 cos\bar{\theta}_3 - 2\dot{\bar{r}}_3\dot{\bar{\theta}}_3 sin\bar{\theta}_3 - \bar{r}_3\left(\ddot{\bar{\theta}}_3 sin\bar{\theta}_3 + \dot{\bar{\theta}}_3^2 cos\bar{\theta}_3\right) \\ -r_5\left(\ddot{\theta}'_5 sin\theta_5 + \dot{\theta}_5^2 cos\theta_5\right) + \ddot{\bar{r}}'_1 cos\bar{\theta}_1 = 0 \\ -r_2 sin\theta_2 + \ddot{\bar{r}}_3 sin\bar{\theta}_3 + 2\dot{\bar{r}}_3\dot{\bar{\theta}}_3 cos\bar{\theta}_3 + \bar{r}_3\left(\ddot{\bar{\theta}}_3 cos\bar{\theta}_3 + \dot{\bar{\theta}}_3^2 sin\bar{\theta}_3\right) \\ +r_5\left(\ddot{\theta}'_5 cos\theta_5 - \dot{\theta}_5^2 sin\theta_5\right) + \ddot{\bar{r}}'_1 sin\bar{\theta}_1 = 0 \end{cases}$$

$$\rightarrow \begin{bmatrix} -r_5 sin\theta_5 & cos\bar{\theta}_1 \\ r_5 cos\theta_5 & sin\bar{\theta}_1 \end{bmatrix} \begin{bmatrix} \ddot{\theta}'_5 \\ \ddot{\bar{r}}'_1 \end{bmatrix} = \begin{bmatrix} r_2\dot{\theta}_2^2 cos\theta_2 - \ddot{\bar{r}}_3 cos\bar{\theta}_3 + 2\dot{\bar{r}}_3\dot{\bar{\theta}}_3 sin\bar{\theta}_3 + \bar{r}_3\left(\ddot{\bar{\theta}}_3 sin\bar{\theta}_3 + \dot{\bar{\theta}}_3^2 cos\bar{\theta}_3\right) \\ +r_5\dot{\theta}_5^2 cos\theta_5 \\ r_2\dot{\theta}_2^2 sin\theta_2 - \ddot{\bar{r}}_3 sin\bar{\theta}_3 - 2\dot{\bar{r}}_3\dot{\bar{\theta}}_3 cos\bar{\theta}_3 - \bar{r}_3\left(\ddot{\bar{\theta}}_3 cos\bar{\theta}_3 - \dot{\bar{\theta}}_3^2 sin\bar{\theta}_3\right) \\ +r_5\dot{\theta}_5^2 sin\theta_5 \end{bmatrix}$$

Solving each coefficient-matrix system gives derivatives with respect to the input angle. These must be converted to time derivatives using the prescribed input angular velocity and acceleration.

$$\dot{r}_3 = \dot{r}'_3 \times \omega_2$$

$$\ddot{r}_3 = \ddot{r}'_3 \times \omega_2^2 + \dot{r}'_3 \times \alpha_2$$

$$\dot{\theta}_3 = \dot{\theta}_3' \times \omega_2$$

$$\ddot{\theta}_3 = \ddot{\theta}_3' \times \omega_2^2 + \dot{\theta}_3' \times \alpha_2$$

$$\dot{\bar{r}}_1 = \dot{\bar{r}}_1' \times \omega_2$$

$$\ddot{\bar{r}}_1 = \ddot{\bar{r}}_1' \times \omega_2^2 + \dot{\bar{r}}_1' \times \alpha_2$$

$$\dot{\theta}_5 = \dot{\theta}_5' \times \omega_2$$

$$\ddot{\theta}_5 = \ddot{\theta}_5' \times \omega_2^2 + \dot{\theta}_5' \times \alpha_2$$

$$\alpha_2 = 0 \frac{rad}{s^2}$$

$$\omega_2 = 1 \frac{rad}{s}$$

3 Dynamic Analysis

Forces on each member are examined, and the input torque required to support a constant load P is calculated.

3.1 Dynamic equations

Equations are written for members 2, 3, 4, 5, and 6. Weight is included in the vertical force balances. The following free-body diagrams and equations retain the source's member and force conventions.

Member 2

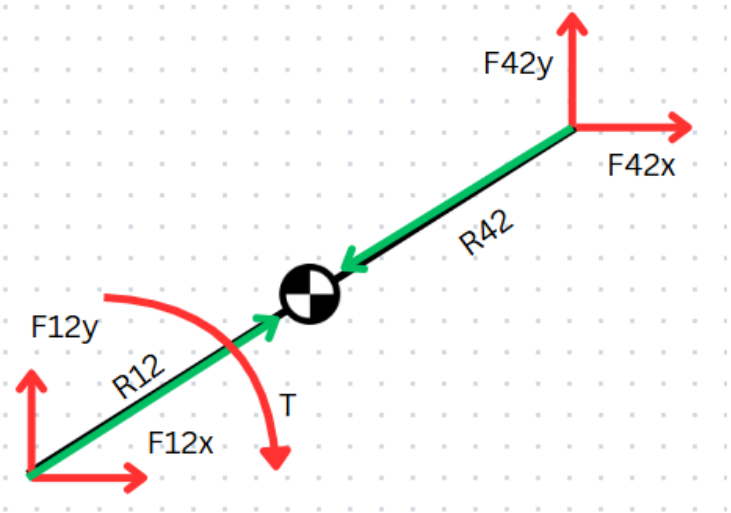


Figure 5: Free-body diagram of member 2.

$$Link2: \begin{cases} F_{12}^x + F_{42}^x = m_2 A_{G_2}^x \\ F_{12}^y + F_{42}^y - w_2 = m_2 A_{G_2}^y \\ -F_{42}^y R_{42}^x + F_{42}^x R_{42}^y + F_{12}^x R_{12}^y - F_{12}^y R_{12}^x + T = I_2 \alpha_2 \end{cases}$$

Member 3

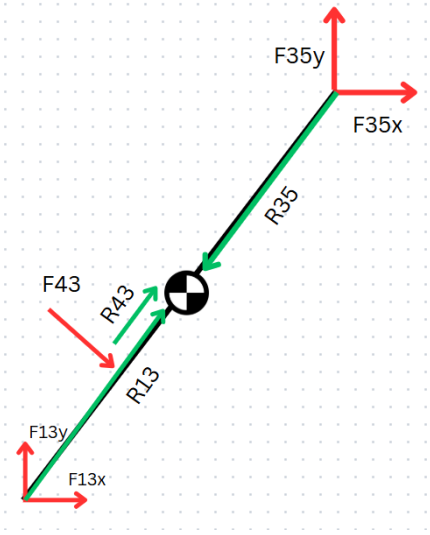


Figure 6: Free-body diagram of member 3.

$$Link3: \begin{cases} F_{13}^x + F_{35}^x + F_{43}^x = m_2 A_{G_3}^x \\ F_{13}^y + F_{35}^y + F_{43}^y - w_3 = m_2 A_{G_3}^y \\ +F_{35}^y R_{35}^x - F_{35}^x R_{35}^y + F_{13}^x R_{13}^y - F_{13}^y R_{13}^x + F_{43}^x R_{43}^y + F_{43}^y R_{43}^x = I_3 \alpha_3 \\ F_{43}^x cos\bar{\theta}_3 - F_{43}^y sin\bar{\theta}_3 = 0 \end{cases}$$

Member 4

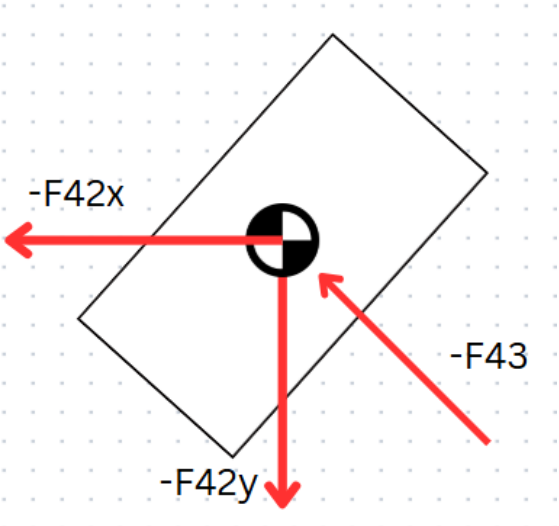


Figure 7: Free-body diagram of slider 4.

$$Link4: \begin{cases} -F_{42}^x - F_{43}^x = m_4 A_{G_4}^x \\ -F_{42}^y + F_{43}^y - w_4 = m_4 A_{G_4}^y \\ 0 + 0 = I_4 \alpha_4 = I_4 \alpha_3 \end{cases}$$

Member 5

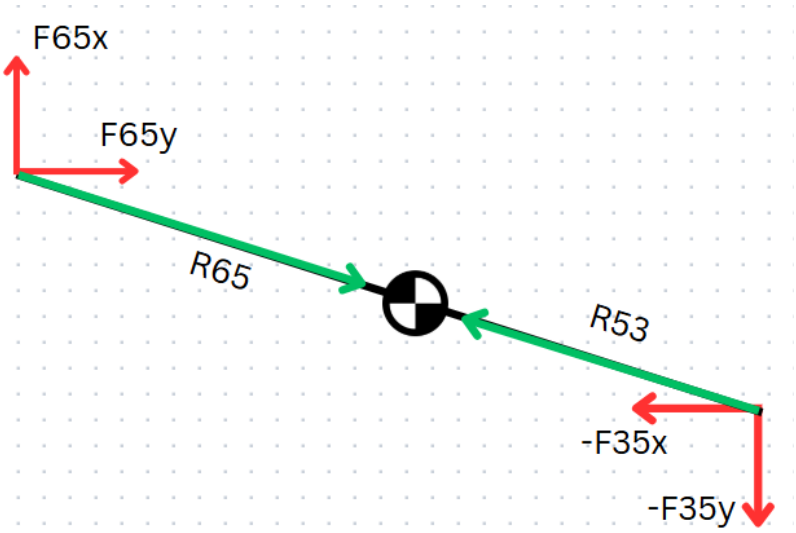


Figure 8: Free-body diagram of member 5.

$$Link5: \begin{cases} -F_{35}^{x} + F_{65}^{x} = m_5 A_{G_5}^{x} \\ -F_{35}^{y} + F_{65}^{y} - w_5 = m_5 A_{G_5}^{y} \\ -F_{35}^{x} R_{35}^{y} - F_{35}^{y} R_{35}^{x} - F_{65}^{x} R_{65}^{y} - F_{65}^{y} R_{65}^{x} = I_5 \alpha_5 \end{cases}$$

Member 6

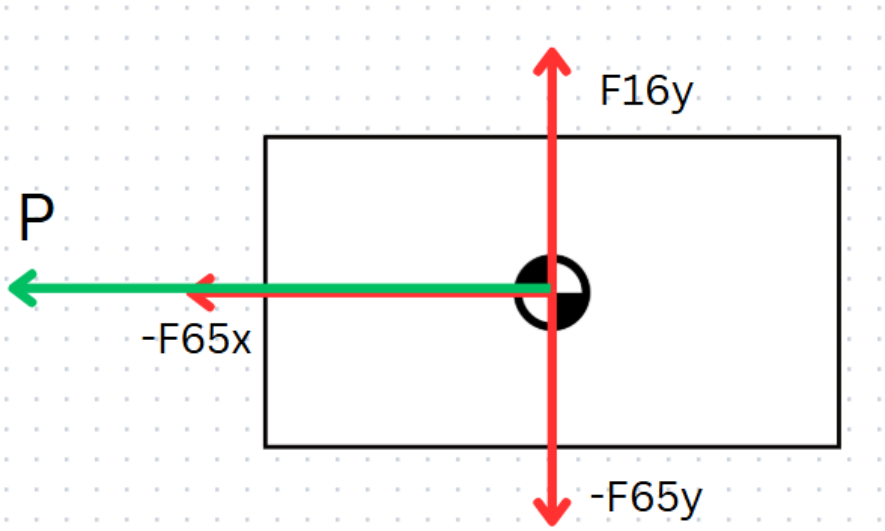


Figure 9: Free-body diagram of slider 6.

$$Link6: \begin{cases} -P - F_{65}^{x} = m_6 A_{G_6}^{x} \\ F_{16}^{y} - F_{65}^{y} - w_6 = 0 \\ 0 + 0 = 0 \end{cases}$$

Member masses and moments of inertia are obtained from Adams. The source assigns a density of 1 kg/m^3 and reads Calculated Mass and IZZ from the information dialog.

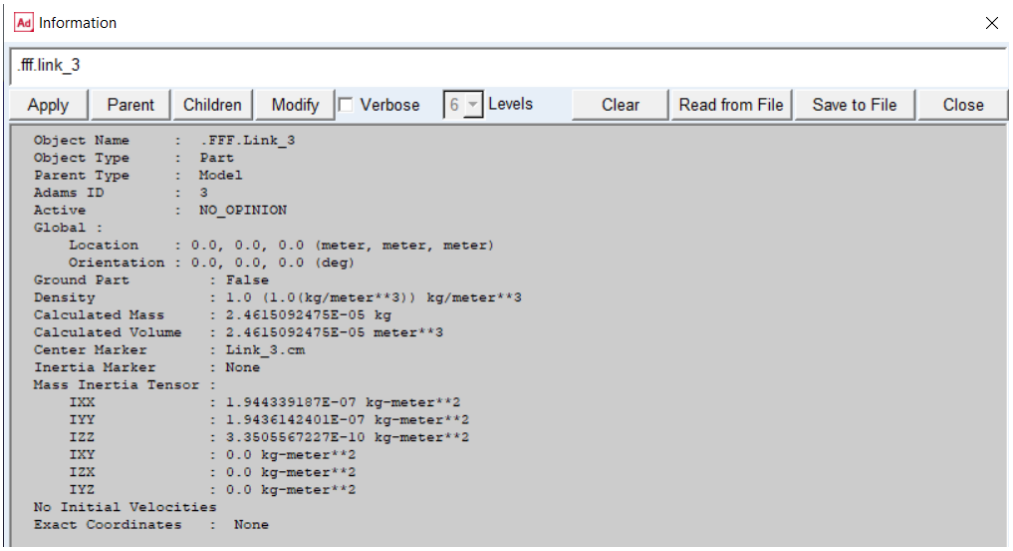


Figure 10: Member properties displayed in Adams.

The dynamic system has fourteen equations and fourteen unknowns. A coefficient matrix is assembled to solve the force and torque components.

$$\begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ R_{12}^y & R_{42}^y & -R_{12}^x & -R_{42}^x & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & R_{13}^y & -R_{35}^y & R_{43}^y & -R_{13}^x & R_{35}^x & R_{43}^x & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & cos\theta_3 & 0 & 0 & -sin\theta_3 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -R_{53}^y & 0 & 0 & -R_{53}^x & 0 & -R_{65}^y & -R_{65}^x & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} F_{12}^x \\ F_{42}^x \\ F_{12}^y \\ F_{42}^y \\ T \\ F_{13}^x \\ F_{35}^x \\ F_{43}^x \\ F_{13}^y \\ F_{35}^y \\ F_{43}^y \\ F_{65}^x \\ F_{65}^y \\ F_{16}^y \end{bmatrix} = \begin{bmatrix} m_2 A_{G_2}^x \\ m_2 A_{G_2}^y + w_2 \\ I_2 \alpha_2 \\ m_3 A_{G_3}^x \\ m_3 A_{G_3}^y + w_3 \\ I_3 \alpha_3 \\ 0 \\ m_4 A_{G_4}^x \\ m_4 A_{G_4}^y + w_4 \\ m_5 A_{G_5}^x \\ m_5 A_{G_5}^y + w_5 \\ I_5 \alpha_5 \\ m_6 A_{G_6}^x + P \\ w_6 \end{bmatrix}$$

The source proposes solving the matrix equation by multiplying the inverse coefficient matrix by the known vector. Because the member-3 angle changes during motion, the system is assembled within a loop to update the angular constraint. Numerical backslash or linear-system solvers are generally preferable to explicitly forming an inverse; the supplied code includes linear solves.

4 Cam Design

Member 2 is removed and replaced with a cam arrangement. The supplied target-angle information, described in the source as spreadsheet data, is used to calculate the cam radius.

4.1 Cam construction

The instantaneous cam radius is determined from the illustrated geometric relationship between the cam and member 3.

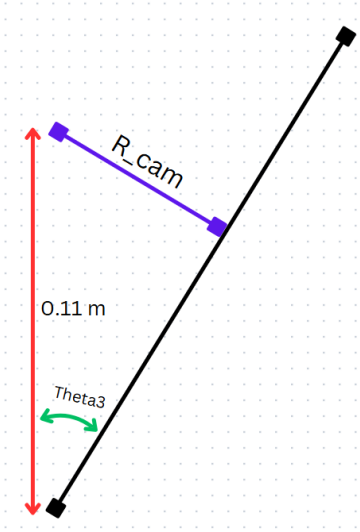


Figure 11: Instantaneous cam and follower geometry.

As the member-3 angle changes at each step, the corresponding cam radius changes. The given angular displacement is used in the following relation.

$$R_{cam} = 0.11 \times sin(\theta_3 + \psi)$$

The radius equation is implemented in Excel to obtain the radius values.

C2 =0.11*SIN(RADIANS(20+B2))

	A	B	C	D	E
1	time	angle (deg)	Cam radius (m)	X	Y
2	0	2.314929499	0.041766695	0.041766695	0.11
3	0.02	2.315755778	0.041768163	0.041764865	0.11052486

Figure 12: Spreadsheet calculation of the cam radius.

The coordinates of the points on the cam profile are then calculated.

D2 =C2*COS(RADIANS(A2*(360/10)))

	A	B	C	D	E
1	time	angle (deg)	Cam radius (m)	X	Y
2	0	2.314929499	0.041766695	0.041766695	0.11
3	0.02	2.315755778	0.041768163	0.041764865	0.11052486
4	0.04	2.317227724	0.041770777	0.041757585	0.111049704
5	0.06	2.319232447	0.041774337	0.041744656	0.111574482
6	0.08	2.321672966	0.041778672	0.041725903	0.112099141

Figure 13: Spreadsheet calculation of cam coordinates.

The X and Y coordinates are saved as CSV data and imported into Adams. The spreadsheet and CSV are not separately included in the supplied archive; the screenshots are retained.

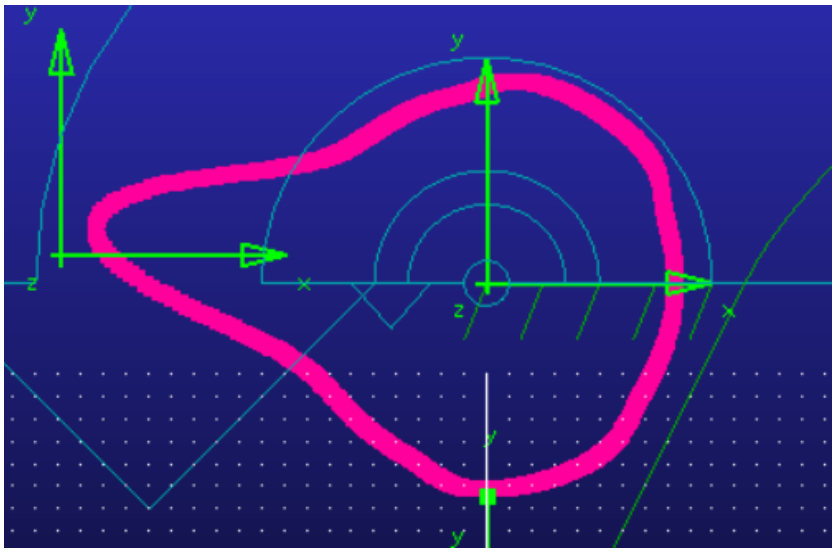


Figure 14: Cam profile imported into Adams.

The cam and member 3 are defined as curves and constrained using a 2D curve–curve connection. MATLAB Curve Fitter is used to approximate the target curve with an R-squared value close to one. The fitted equation is then entered into Excel to generate coordinates for the Adams geometry. The source describes the simulated follower motion as resembling the project target. A numerical error metric is not supplied.

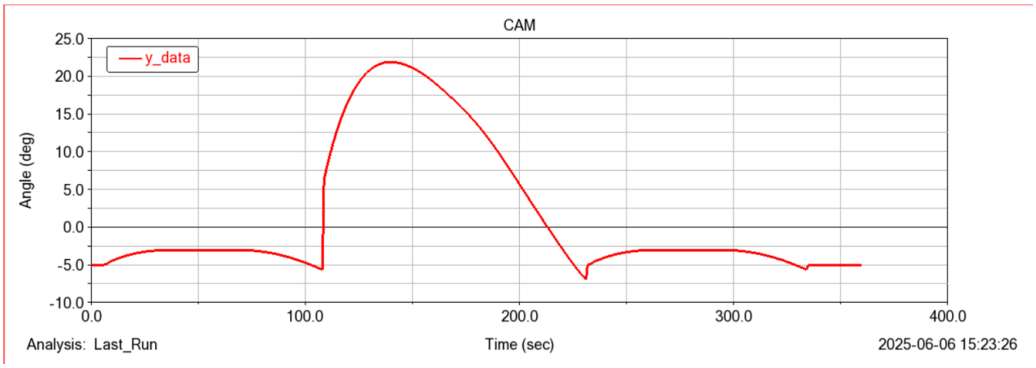


Figure 15: Follower angle obtained using the cam in Adams.

5 Simulation and Comparison

This section describes the Adams, SolidWorks, and MATLAB modeling and comparison procedure.

5.1 Adams model

The initial mechanism geometry is tested in Adams by trying different point locations until the arrangement shown below is obtained.

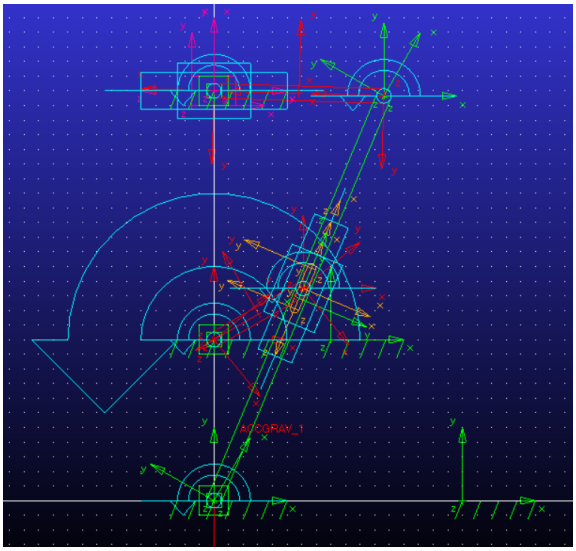


Figure 16: Quick-return mechanism in Adams.

The source describes prescribing crank rotation with a one-degree time-based increment and running two revolutions with 500 simulation steps. Members 2 and 4 are pinned together, while members 3 and 4 have relative sliding motion. Member 5 is pinned to member 6, and member 6 slides relative to ground. Measure definitions are used to track link angles and vector lengths over time. The exact motion-function units should be checked in the native model.

5.2 Comparison plots

The following plots compare the appended MATLAB analysis with Adams over two crank revolutions.

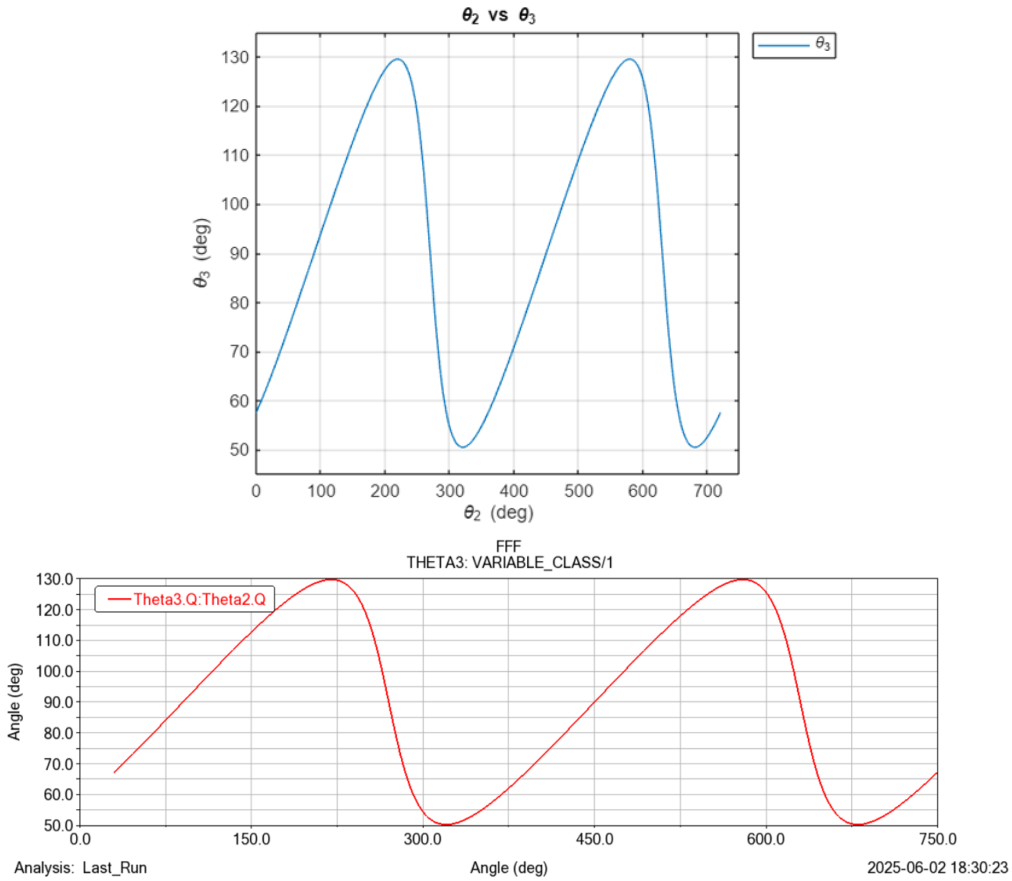


Figure 17: Member 3 angular-position comparison.

The source describes the member-3 angle plots as similar. Their different initial phase is attributed to MATLAB starting the crank at zero degrees and Adams starting at 30 degrees.

The member-5 angle plots are also described as similar, with the initial-angle difference discussed above.

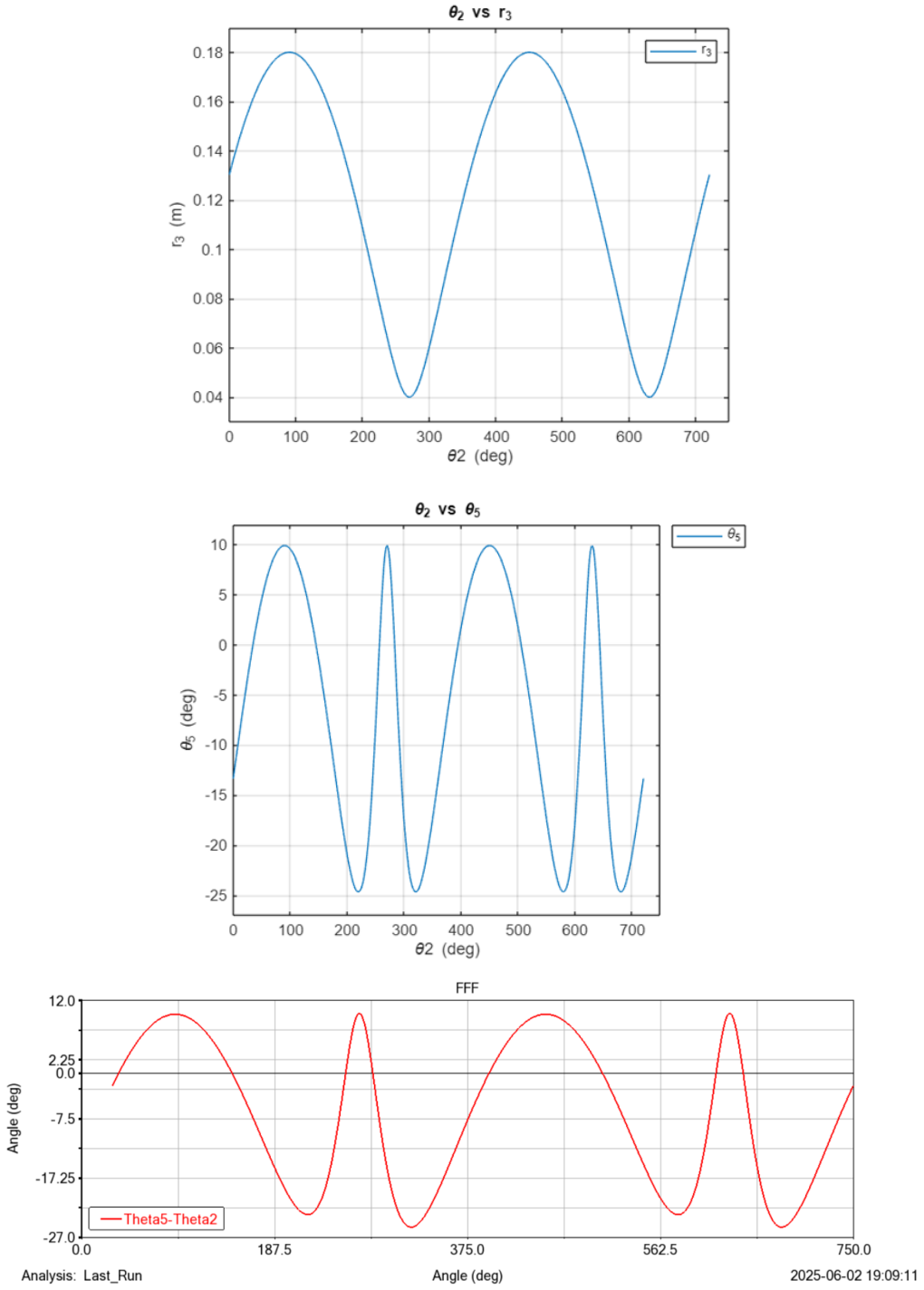


Figure 18: Original figures accompanying the member 5 angle comparison.

The variable vector-length plot represents the distance from slider 4 to the O3 support.

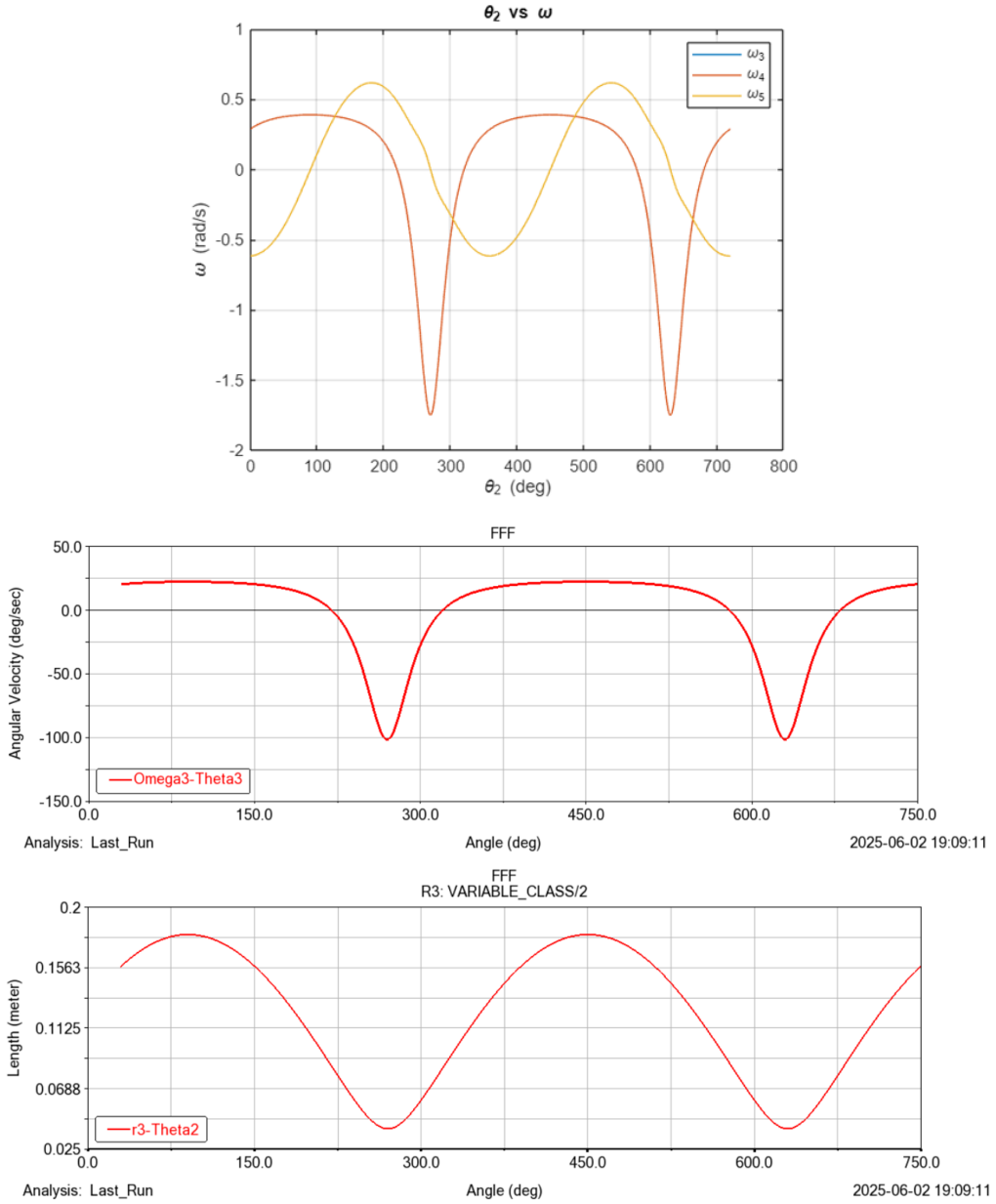


Figure 19: Original figures accompanying the variable-length comparison.

The member-3 angular velocities are described as similar, but member 5's angular velocity does not agree. The source suspects an error in assembling the MATLAB equations. It notes that the following acceleration comparison appears similar, but that observation alone does not prove the velocity discrepancy has been resolved. The member-3 and member-4 curves overlap in the MATLAB velocity plot because those members share angular motion; the same observation is made for angular acceleration.

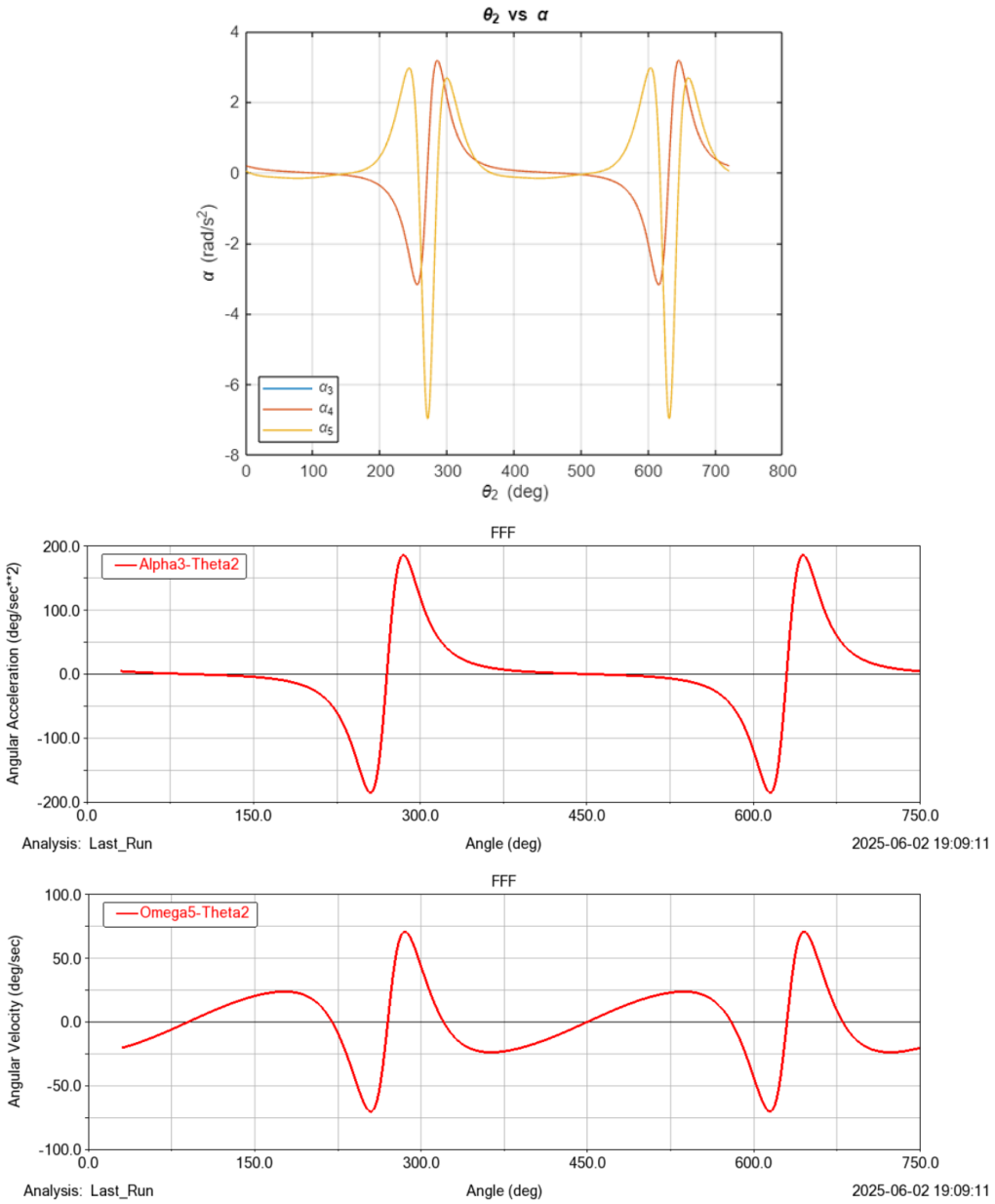


Figure 20: Angular-velocity comparison figures.

The source states that the angular-acceleration plots appear similar despite the discrepancy in member-5 angular velocity. Original figures are retained as supplied; one figure in this location is labeled as a force plot rather than an angular-acceleration plot.

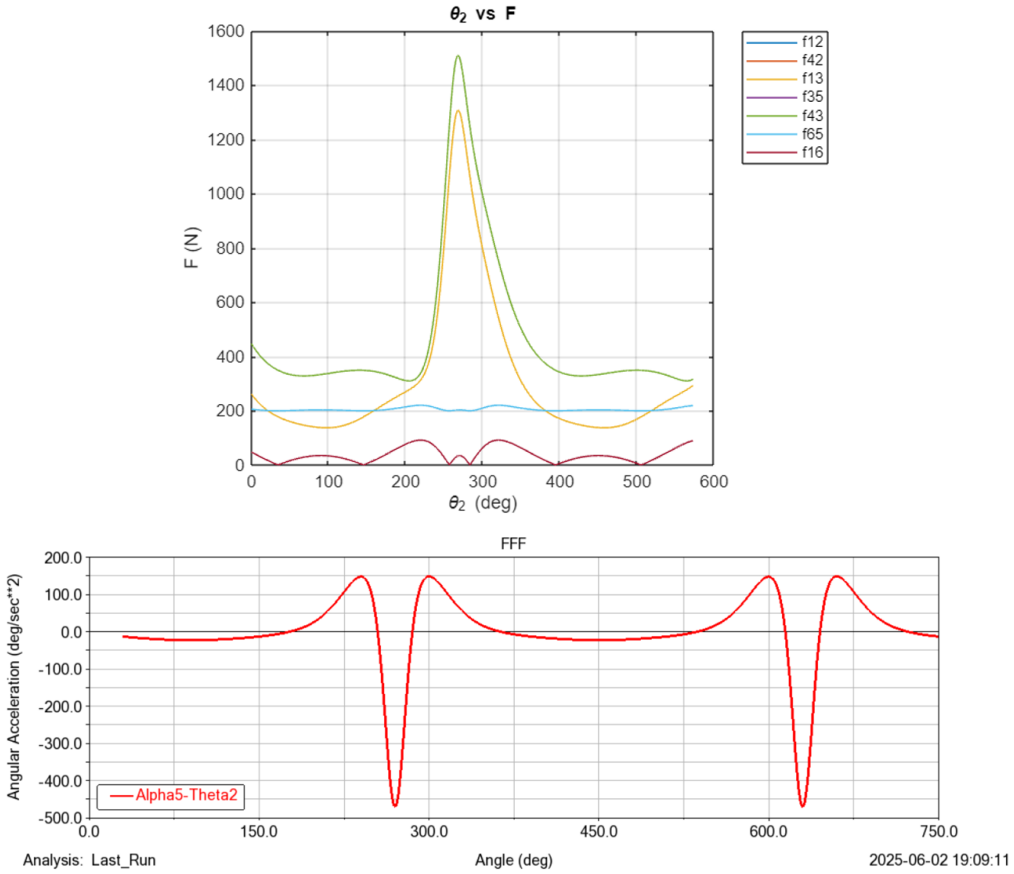


Figure 21: Original figures accompanying the acceleration comparison; one is labeled as a force plot.

The following discussion concerns the force histories over the nominal ten-second interval. The source places the corresponding plots in its appendix; they are reproduced there in this edition.

The source identifies the largest forces in the approximate crank-angle range 250–350 degrees, where output slider 6 is pulled rapidly backward and decelerates. It attributes the force increase to this change in motion and describes slider 6 as the heaviest member. The saved mass assumptions should be checked before adopting that description.

The source attributes the input-torque peak to the same motion phase as the force increase.

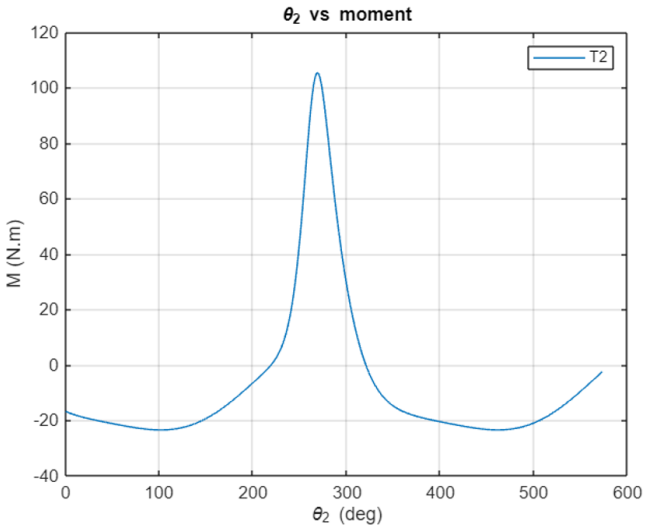


Figure 22: Input-torque history.

A plot is also obtained for the designed cam radius.

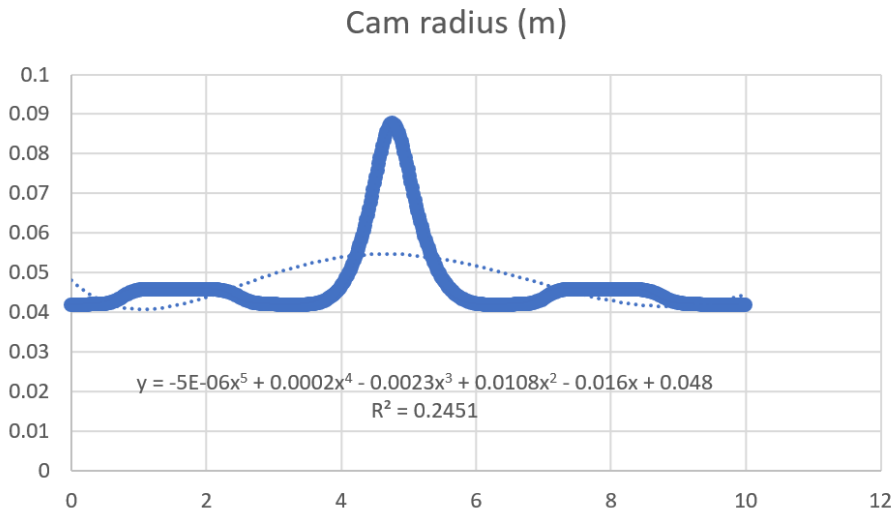


Figure 23: Cam-radius plot.

5.3 SolidWorks design

A SolidWorks model is created to determine initial dimensions and angles and to visualize the mechanism more clearly.

A ground support is included to make the assembly more representative. The holes in the links represent pin joints and have a diameter of 8 mm.

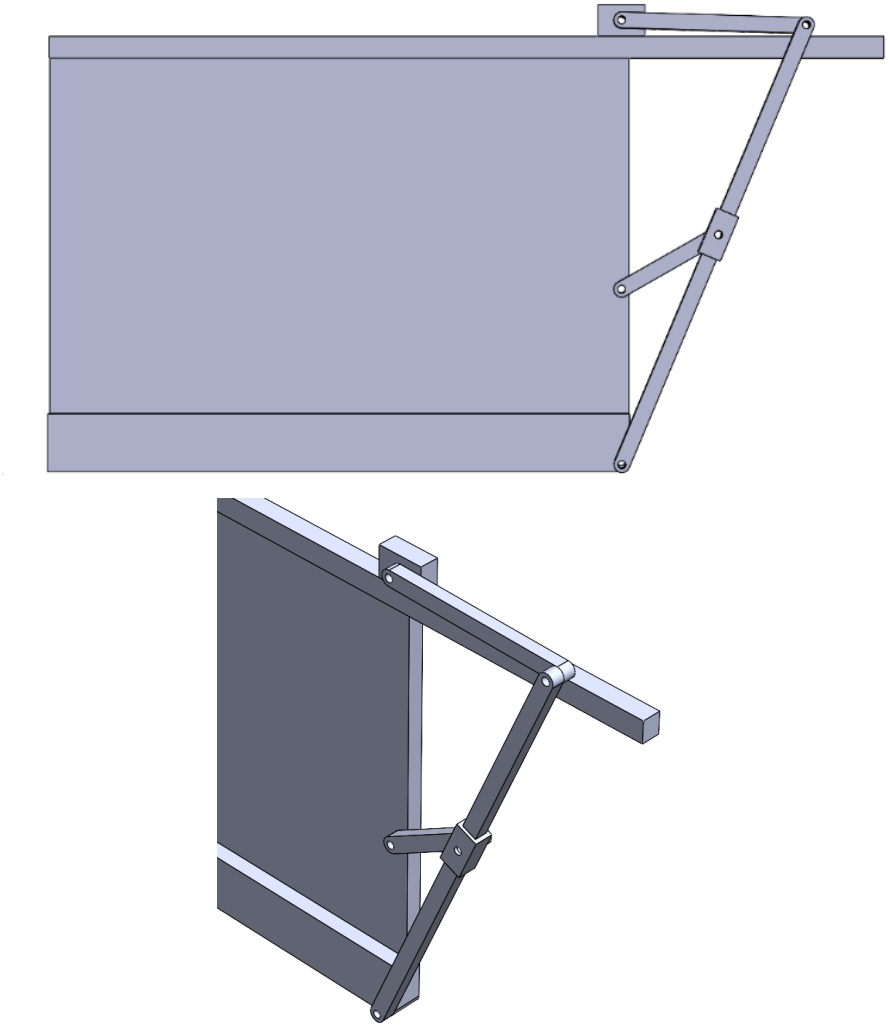


Figure 24: SolidWorks mechanism model.

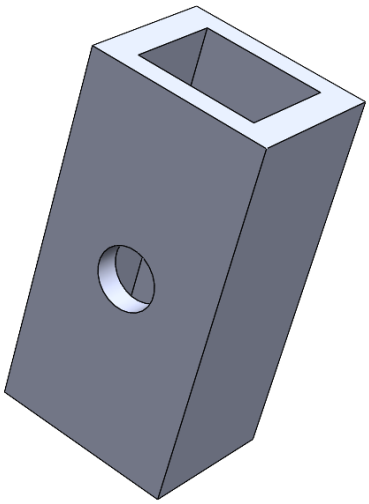


Figure 25: Slider 4 geometry.

The slider opening is 10 by 8 mm, matching the cross section of member 3 along which it slides. An animation of this assembly is included in the archive. Manufacturing clearances are not established by the nominal dimensions alone.

6 Conclusion

The project examines the kinematic and dynamic behavior of a quick-return mechanism through analytical modeling and software simulation. A MATLAB program calculates and plots angular velocities and accelerations. The source concludes that the selected geometry supports complete continuous motion.

Joint forces and input torque are calculated for the stated loading conditions. The source interprets the results as predictable operation of the mechanism and describes the cam-based alternative as reproducing the desired motion. These are the author's interpretations of the software study, subject to the unresolved comparison and model issues noted in this edition.

Quick-return mechanisms offer a simple way to obtain controlled reciprocating motion. Appropriate design may reduce cycle time and improve machine operation, although this project does not provide a quantified energy-efficiency optimization.

The study illustrates the use of analytical methods alongside simulation in mechanism design. Future work could examine friction and vibration to obtain a more realistic model.

7 References

Machine Dynamics teaching-assistant project instructions, School of Mechanical Engineering. Not supplied as a separate file.

Robert L. Norton, Design of Machinery, third edition, McGraw-Hill, 2004. Citation retained from the source report.

A MATLAB Source Listing

The listing below is extracted from the original report. Section headings are comments; the numerical logic is retained. The native Live Script is supplied separately and may contain a different saved revision.

```
clear all
clc

%% INPUT
% Force
p=200; % N
% lengths determined by trial and error in ADAMS
r1=0.11;
r2=0.07;
r5=0.1164;
r1_bar=0.17;
O3C=0.3;



%Constant angles
theta1=pi/2;
theta1_bar=1.5*pi;
theta1_2bar=0;

```

```
N=2000;
theta2=linspace(0,4*pi,N);

% Initial conditions for theta_2 = 0 (Found from SOLIDWORKS)
theta3_1=deg2rad(180+57);
theta3bar_1=deg2rad(57);
theta5_1=deg2rad(166);
r1_2bar_1=0;
r3_1=0.13038;
r3_bar_1=0.3-r3_1;

w2=1 ; % 1 rad/sec : input angular velocity
a2=0; %input angular acceleration (=0 Given Data)

%% NEWTON-RAPHSON CONTROLLERS
maxiter=100;
res=0.0001;

function [x,y,loop_break] = newtonraphson(f,g,jacob,x,y,maxiter,res,x1,y1)
delta=inf(2,1);
z=[x1;y1];
iter=1;
loop_break=false;
while abs(delta(1,1))>res || abs(delta(2,1))>res
    delta=double(-jacob(x1,y1)\[f(x1,y1);g(x1,y1)]);
    z=z+delta;
    x1=z(1,1);
    y1=z(2,1);
    iter=iter+1;
    if iter>maxiter
        disp('issue')
        loop_break=true;
        break
    end
end
x=z(1,1);
y=z(2,1);
end

%% POSTURE
syms Theta2 Theta3 R3 Theta5 R1_2b Theta3_b r3_b


f1(Theta3,R3,Theta2)=r2*cos(Theta2)+R3*cos(Theta3)+r1*cos(theta1);
g1(Theta3,R3,Theta2)=r2*sin(Theta2)+R3*sin(Theta3)+r1*sin(theta1);
jacob1(Theta3,R3,Theta2)=[diff(f1,Theta3),diff(f1,R3);diff(g1,Theta3),diff(g1,R3)];

f1_func = matlabFunction(f1, 'Vars', [Theta3,R3,Theta2]);
g1_func = matlabFunction(g1, 'Vars', [Theta3,R3,Theta2]);
jacob1_func = matlabFunction(jacob1, 'Vars', [Theta3,R3,Theta2]);


f2(Theta5,R1_2b,R3,Theta3,Theta2)=r2*cos(Theta2)+(O3C-R3)*cos(Theta3-pi)+r5*cos(Theta5)+r1_bar*cos(
    theta1_bar)+R1_2b*cos(theta1_2bar);
g2(Theta5,R1_2b,R3,Theta3,Theta2)=r2*sin(Theta2)+(O3C-R3)*sin(Theta3-pi)+r5*sin(Theta5)+r1_bar*sin(
    theta1_bar)+R1_2b*sin(theta1_2bar);
jacob2(Theta5,R1_2b,Theta2)=[diff(f2,Theta5),diff(f2,R1_2b);diff(g2,Theta5),diff(g2,R1_2b)];

f2_func = matlabFunction(f2, 'Vars', [Theta5,R1_2b,R3,Theta3,Theta2]);
g2_func = matlabFunction(g2, 'Vars', [Theta5,R1_2b,R3,Theta3,Theta2]);
jacob2_func = matlabFunction(jacob2, 'Vars', [Theta5,R1_2b,Theta2]);

r3=zeros(N,1);
teta3=zeros(N,1);
teta5=zeros(N,1);
r1_2b=zeros(N,1);

for i=1:N

    %loop1
    f1_handle = @(Theta3, R3) f1_func(Theta3, R3, theta2(i));
    g1_handle = @(Theta3, R3) g1_func(Theta3, R3, theta2(i));
    jacob1_handle = @(Theta3, R3) jacob1_func(Theta3, R3, theta2(i));
```

```
92      [teta_3,r_3,loop_break] = newtonraphson(f1_handle,g1_handle,jacob1_handle,Theta3,R3,maxiter,res,
        theta3_1,r3_1);
93      teta3(i)=teta_3;
94      r3(i)=r_3;
95      theta3_1=teta_3;
96      r3_1=r_3;
97      if loop_break, break, end
98
99      %loop2
100     f2_handle = @(Theta5,R1_2b) f2_func(Theta5,R1_2b,r3(i),teta3(i),theta2(i));
101     g2_handle = @(Theta5,R1_2b) g2_func(Theta5,R1_2b,r3(i),teta3(i),theta2(i));
102     jacob2_handle = @(Theta5,R1_2b) jacob2_func(Theta5,R1_2b,theta2(i));
103     [teta_5,r_1_2b,loop_break] = newtonraphson(f2_handle,g2_handle,jacob2_handle,Theta5,R1_2b,maxiter,res,
        theta5_1,r1_2bar_1);
104     teta5(i)=teta_5;
105     r1_2b(i)=r_1_2b;
106     theta5_1=teta_5;
107     r1_2bar_1=r_1_2b;
108     if loop_break, break, end
109
110 end
111
112 %% POSITION OF EACH POINT
113 p_O3=[0,0,0];
114 p_O2=[0,0.11,0];
115 p_B=zeros(N,3);
116 p_C=zeros(N,3);
117 p_D=zeros(N,3);
118
119 p_G2=zeros(N,3);
120 p_G3=zeros(N,3);
121 p_G4=zeros(N,3);
122 p_G5=zeros(N,3);
123 p_G6=zeros(N,3);
124
125 for i=1:N
126     p_B(i,:)=p_O2+r2*[cos(theta2(i)),sin(theta2(i)),0];
127     p_C(i,:)=p_O3+O3C*[cos(teta3(i)-pi),sin(teta3(i)-pi),0];
128     p_D(i,:)=p_C(i,:)+r5*[cos(teta5(i)),sin(teta5(i)),0];
129
130     p_G2(i,:)=p_O2+r2/2*[cos(theta2(i)),sin(theta2(i)),0];
131     p_G3(i,:)=p_O3+O3C/2*[cos(teta3(i)-pi),sin(teta3(i)-pi),0];
132     p_G4(i,:)=p_B(i,:);
133     p_G5(i,:)=p_C(i,:)+r5/2*[cos(teta5(i)-pi),sin(teta5(i)-pi),0];
134     p_G6(i,:)=p_D(i,:);
135 end
136
137 %% VELOCITY & ACCELERATION
138 teta3p=zeros(N,1);
139 r3p=zeros(N,1);
140 teta5p=zeros(N,1);
141 r1_2bp=zeros(N,1);
142
143 teta3pp=zeros(N,1);
144 r3pp=zeros(N,1);
145 teta5pp=zeros(N,1);
146 r1_2bpp=zeros(N,1);
147
148 for i=1:N
149 %velocity
150     %loop1
151     A1=[cos(teta3(i)),-r3(i)*sin(teta3(i));sin(teta3(i)),r3(i)*cos(teta3(i))];
152     B1=r2*[sin(theta2(i));-cos(theta2(i))];
153     v1=A1\B1;
154     if det(A1)==0
155         disp('singular v1');
156     end
157     teta3p(i)=v1(2);
158     r3p(i)=v1(1);
159
160     %loop2
161
162     A2=[-r5*sin(teta5(i)),cos(theta1_2bar);r5*cos(teta5(i)),sin(theta1_2bar)];
```

```
    B2=[r2*sin(theta2(i)+r3p(i)*cos(teta3(i)-pi)+(O3C-r3(i))*teta3p(i)*sin(teta3(i)-pi));...
        r2*cos(theta2(i)-r3p(i)*sin(teta3(i)-pi)-(O3C-r3(i))*teta3p(i)*cos(teta3(i)-pi))]; %theta3p =
     theta3barp /// r3p=-r3barp
    v2=A2\B2;
    if det(A2)==0
        disp('singular v2');
    end
    teta5p(i)=v2(1);
    r1_2bp(i)=v2(2);

%acceleration
    %loop1
    C1=r2*[cos(theta2(i));sin(theta2(i))] + r3p(i)*teta3p(i)^2*[cos(teta3(i));sin(teta3(i))] + 2*r3p(i)*
     teta3p(i)*[sin(teta3(i));-cos(teta3(i))];
    Ac1=A1\C1;
    teta3pp(i)=Ac1(2);
    r3pp(i)=Ac1(1);
    %loop2
    C2=[r2*cos(theta2(i))+r3pp(i)*cos(teta3(i)-pi)-2*r3p(i)*teta3p(i)*sin(teta3(i)-pi)+(O3C-r3(i))*(teta3pp
     (i)*sin(teta3(i)-pi)+(teta3p(i))^2*cos(teta3(i)-pi))+r5*teta5p(i)^2*cos(teta5(i));...
        r2*sin(theta2(i))+r3pp(i)*sin(teta3(i)-pi)+2*r3p(i)*teta3p(i)*cos(teta3(i)-pi)-(O3C-r3(i))*(teta3pp
     (i)*cos(teta3(i)-pi)-(teta3p(i))^2*sin(teta3(i)-pi))+r5*teta5p(i)^2*sin(teta5(i))];
    Ac2=A2\C2;
    teta5pp(i)=Ac2(1);
    r1_2bpp(i)=Ac2(2);
end

w3=w2.*teta3p;
w4=w3;
w5=w2.*teta5p;
v6=w2.*r1_2bp;

a3=w2.^2.*teta3pp+a2.*teta3p;
a4=a3;
a5=w2.^2.*teta5pp+a2.*teta5p;
A6=w2.^2.*r1_2bpp+a2.*r1_2bp;

a_G2=zeros(N,3);
a_G3=zeros(N,3);
a_G4=zeros(N,3);
a_G5=zeros(N,3);
a_G6=zeros(N,3);

aD=zeros(N,3);
aB=zeros(N,3);
aC=zeros(N,3);
for i=1:N
    a_G2(i, :)=cross([0,0,a2],p_G2(i, :)-p_O2)+cross([0,0,w2],cross([0,0,w2],p_G2(i, :)-p_O2));
    aB(i, :)=cross([0,0,a2],p_B(i, :)-p_O2)+cross([0,0,w2],cross([0,0,w2],p_B(i, :)-p_O2));
    a_G3(i, :)=cross([0,0,a3(i)],p_G3(i, :)-p_O3)+cross([0,0,w3(i)],cross([0,0,w3(i)],p_G3(i, :)-p_O3));
    a_G4(i, :)=aB(i, :);
    aC(i, :)=cross([0,0,a3(i)],p_C(i, :)-p_O3)+cross([0,0,w3(i)],cross([0,0,w3(i)],p_C(i, :)-p_O3));
    a_G5(i, :)=aC(i, :)+cross([0,0,a5(i)],p_G5(i, :)-p_C(i, :))+cross([0,0,w5(i)],cross([0,0,w5(i)],p_G5(i,
      :)-p_C(i, :)));
    aD(i, :)=aC(i, :)+cross([0,0,a5(i)],p_D(i, :)-p_C(i, :))+cross([0,0,w5(i)],cross([0,0,w5(i)],p_D(i, :)-
     p_C(i, :)));
    a_G6(i, :)=aD(i, :);
end


%% DYNAMIC FORCE EQUATIONS
syms f12x f12y f42x f42y T f13x f13y f35x f35y f43x f43y f65x f65y f16y
syms aG2x aG2y aG3x aG3y aG4x aG4y aG5x aG5y aG6x aG6y
syms alfa2 alfa3 alfa4 alfa5 alfa6
syms P_O2x P_O2y P_Bx P_By P_Cx P_Cy P_O3x P_O3y P_Dx P_Dy
syms P_G2x P_G2y P_G3x P_G3y P_G4x P_G4y P_G5x P_G5y P_G6x P_G6y
var=[f12x f12y f42x f42y T f13x f13y f35x f35y f43x f43y f65x f65y f16y];
unknowns = zeros(numel(var), 10);

P_O2 = [P_O2x,P_O2y,0];
P_B = [P_Bx,P_By,0];
P_C = [P_Cx,P_Cy,0];
P_D = [P_Dx,P_Dy,0];
```

```
P_O3 = [P_O3x,P_O3y,0];

P_G2 = [P_G2x,P_G2y,0];
P_G3 = [P_G3x,P_G3y,0];
P_G4 = [P_G4x,P_G4y,0];
P_G5 = [P_G5x,P_G5y,0];
P_G6 = [P_G6x,P_G6y,0];

% rho=1kg/m^3
m2=6.244/10^6;
m3=2.4615/10^5;
m4=1.6603/10^6;
m5=9.9262/10^6;
m6=2.4/10^5;
g=9.81;
W2=m2*g;
W3=m3*g;
W4=m4*g;
W5=m5*g;
W6=m6*g;

% Izz from Adams
I2=8.399/10^11;
I3=3.35/10^10;
I4=1.336/10^11;
I5=1.343/10^10;
I6=2.6/10^9;

e = sym(zeros(14, 1));
% m=100;
m=(10*2000)/(4*pi);
for i=1:m
    % Data:
    % P_O2x = p_O2(1);
    % P_O2y = p_O2(2);
    % P_Bx = p_B(i,1);
    % P_By = p_B(i,2);
    % P_Cx = p_C(i,1);
    % P_Cy = p_C(i,2);
    % P_O3x = p_O3(1);
    % P_O3y = p_O3(2);
    % P_Dx = p_D(i,1);
    % P_Dy = p_D(i,2);
    %
    % P_G2x = p_G2(i,1);
    % P_G2y = p_G2(i,2);
    % P_G3x = p_G3(i,1);
    % P_G3y = p_G3(i,2);
    % P_G4x = p_G4(i,1);
    % P_G4y = p_G4(i,2);
    % P_G5x = p_G5(i,1);
    % P_G5y = p_G5(i,2);
    % P_G6x = p_G6(i,1);
    % P_G6y = p_G6(i,2);
    %
    % aG2x=a_G2(i,1);
    % aG2y=a_G2(i,2);
    % aG3x=a_G3(i,1);
    % aG3y=a_G3(i,2);
    % aG4x=a_G4(i,1);
    % aG4y=a_G4(i,2);
    % aG5x=a_G5(i,1);
    % aG5y=a_G5(i,2);
    % aG6x=a_G6(i,1);
    % aG6y=a_G6(i,2);
    %
    % alfa2=a2;
    % alfa3=a3(i);
    % alfa4=alfa3;
    % alfa5=a5(i);
    % alfa6=0;


```

```
% 13 equations, 13 unknowns

    %part 2
    e(1)=m2*aG2x==f12x+f42x;
    e(2)=m2*aG2y==f12y+f42y-W2;
    M2=cross(P_O2-P_G2,[f12x,f12y,0])+cross(P_B-P_G2,[f42x,f42y,0])+[0,0,T];
    e(3)=I2*alfa2==M2(3);
    %part 3
    e(4)=m3*aG3x==f13x+f35x+f43x;
    e(5)=m3*aG3y==f13y+f35y-f43y-W3;
    M3=cross(P_O3-P_G3,[f13x,f13y,0])+cross(P_C-P_G3,[f35x,f35y,0])+cross(P_G4-P_G3,[f43x,-f43y,0]);
    e(6)=I3*alfa3==M3(3);
    e(14)=f43x*cos(teta3(i)-pi)-f43y*sin(teta3(i)-pi)==0;
    %part 4
    e(7)=m4*aG4x==-f42x-f43x;
    e(8)=m4*aG4y==-f42y+f43y-W4;
    %part 5
    e(9)=m5*aG4x==-f35x+f65x;
    e(10)=m5*aG4y==-f35y+f65y-W5;
    M5=cross(P_C-P_G5,-[f35x,f35y,0])+cross(P_D-P_G5,[f65x,f65y,0]);
    e(11)=I5*alfa5==M5(3);
    %part 6
    e(12)=m6*aG6x==-f65x-p;
    e(13)=m6*aG6y==-f65y+f16y-W6;



    [A_sym, B_sym] = equationsToMatrix(e, var);


     A = subs(A_sym, {
        P_O2x, P_O2y, P_Bx, P_By, P_Cx, P_Cy, P_O3x, P_O3y, P_Dx, P_Dy, ...
        P_G2x, P_G2y, P_G3x, P_G3y, P_G4x, P_G4y, P_G5x, P_G5y, P_G6x, P_G6y, ...
        aG2x, aG2y, aG3x, aG3y, aG4x, aG4y, aG5x, aG5y, aG6x, aG6y, ...
        alfa2, alfa3, alfa4, alfa5, alfa6
    }, {
        p_O2(1), p_O2(2), p_B(i,1), p_B(i,2), p_C(i,1), p_C(i,2), p_O3(1), p_O3(2), p_D(i,1), p_D(i,2), ...
        p_G2(i,1), p_G2(i,2), p_G3(i,1), p_G3(i,2), p_G4(i,1), p_G4(i,2), p_G5(i,1), p_G5(i,2), p_G6(i,1),
     p_G6(i,2), ...
        a_G2(i,1), a_G2(i,2), a_G3(i,1), a_G3(i,2), a_G4(i,1), a_G4(i,2), a_G5(i,1), a_G5(i,2), a_G6(i,1),
     a_G6(i,2), ...
        a2, a3(i), a3(i), a5(i), 0
    });

    % unknowns(:, i) = A_sym \ B_sym;
    u_sym= linsolve(A,B_sym);

    unknowns(:,i)= subs(u_sym, {
        P_O2x, P_O2y, P_Bx, P_By, P_Cx, P_Cy, P_O3x, P_O3y, P_Dx, P_Dy, ...
        P_G2x, P_G2y, P_G3x, P_G3y, P_G4x, P_G4y, P_G5x, P_G5y, P_G6x, P_G6y, ...
        aG2x, aG2y, aG3x, aG3y, aG4x, aG4y, aG5x, aG5y, aG6x, aG6y, ...
        alfa2, alfa3, alfa4, alfa5, alfa6
    }, {
        p_O2(1), p_O2(2), p_B(i,1), p_B(i,2), p_C(i,1), p_C(i,2), p_O3(1), p_O3(2), p_D(i,1), p_D(i,2), ...
        p_G2(i,1), p_G2(i,2), p_G3(i,1), p_G3(i,2), p_G4(i,1), p_G4(i,2), p_G5(i,1), p_G5(i,2), p_G6(i,1),
     p_G6(i,2), ...
        a_G2(i,1), a_G2(i,2), a_G3(i,1), a_G3(i,2), a_G4(i,1), a_G4(i,2), a_G5(i,1), a_G5(i,2), a_G6(i,1),
     a_G6(i,2), ...
        a2, a3(i), a3(i), a5(i), 0
    });
    % unknowns(:, i) =linsolve(A_sym,B_sym)
    % unknowns(:, i)=double(u_sym(i));
end
for i = 1:length(var)
    var_name = char(var(i));
    eval([var_name ' = double(unknowns(i, :));']);
end
f12=sqrt(f12x.^2+f12y.^2);
f42=sqrt(f42x.^2+f42y.^2);
f13=sqrt(f13x.^2+f13y.^2);
f35=sqrt(f35x.^2+f35y.^2);
f16=sqrt(f16y.^2);
f65=sqrt(f65x.^2+f65y.^2);
```

```
f43=sqrt(f43x.^2+f43y.^2);

%% PLOT
figure;
plot(rad2deg(theta2),rad2deg(teta3-pi))
xlim([0, 750]);
ylim([45, 135]);
grid on
legend('\theta_3','Location','northeastoutside')
title('\theta_2 vs \theta_3')
xlabel("\theta_2 (deg)")
ylabel("\theta_3 (deg)")




figure;
plot(rad2deg(theta2),rad2deg(teta5-pi))
xlim([0, 750]);
ylim([-27, 12]);
grid on
legend('\theta_5','Location','northeastoutside')
title('\theta_2 vs \theta_5')
xlabel("\theta2 (deg)")
ylabel("\theta_5 (deg)")




figure;
plot(rad2deg(theta2),r3);
xlim([0, 750]);
ylim([0.03 0.19]);
grid on;
legend('r_3')
title('\theta_2 vs r_3')
xlabel("\theta2 (deg)")
ylabel("r_3 (m)")




figure;
plot(rad2deg(theta2),w3)
hold on
plot(rad2deg(theta2),w4)
hold on
plot(rad2deg(theta2),w5)
hold off
grid on
legend('\omega_3','\omega_4','\omega_5')
title('\theta_2 vs \omega')
xlabel("\theta_2 (deg)")
ylabel("\omega (rad/s)")




figure;
plot(rad2deg(theta2),a3)
hold on
plot(rad2deg(theta2),a4)
hold on
plot(rad2deg(theta2),a5)
hold off
grid on
```

```
legend('\alpha_3','\alpha_4','\alpha_5','Location','southwest')
title('\theta_2 vs \alpha')
xlabel("\theta_2 (deg)")
ylabel("\alpha (rad/s^2)")









figure;
m_index=1591;
plot(rad2deg(theta2(1:m_index)),f12)
hold on
plot(rad2deg(theta2(1:m_index)),f42)
hold on
plot(rad2deg(theta2(1:m_index)),f13)
hold on
plot(rad2deg(theta2(1:m_index)),f35)
hold on
plot(rad2deg(theta2(1:m_index)),f43)
hold on
plot(rad2deg(theta2(1:m_index)),f65)
hold on
plot(rad2deg(theta2(1:m_index)),f16)
hold off
grid on
legend('f12','f42','f13','f35','f43','f65','f16','Location','northeastoutside')
title('\theta_2 vs F')
xlabel("\theta_2 (deg)")
ylabel("F (N)")









figure;
plot(rad2deg(theta2(1:m_index)),T)
grid on
legend('T')
title('\theta_2 vs moment')
xlabel("\theta_2 (deg)")
ylabel("M (N.m)")









%% ANIMATION
figure;
axis equal;
grid on;
hold on;
Xlabel='X Position (m)';
Ylabel='Y Position (m)';
xlabel(Xlabel, 'FontName', 'Times New Roman', 'FontSize', 24, 'FontWeight', 'bold');
ylabel(Ylabel, 'FontName', 'Times New Roman', 'FontSize', 24, 'FontWeight', 'bold');
Title='Animation';
title(Title, 'FontName', 'Times New Roman', 'FontSize', 30, 'FontWeight', 'bold');
set(gca, 'FontSize', 20, 'FontName', 'Times New Roman', 'FontWeight', 'bold');

xlim([-0.35, 0.25]);
ylim([-0.05, 0.35]);

l2_plot = plot(0, 0, 'b', 'LineWidth', 2);
l3_plot = plot(0, 0, 'g', 'LineWidth', 2);
l4_plot = plot(0, 0, 'r', 'LineWidth', 5);
l5_plot = plot(0, 0, 'c', 'LineWidth', 2);

rectWidth4 = 0.03; % Width of slider 4
rectHeight4 = 0.015; % Height of slider 4
```

```
518
519 rectWidth6 = 0.03; % Width of slider 4
520 rectHeight6 = 0.02; % Height of slider 4
521
522 rect_plot = rectangle('Position', [0, 0, rectWidth6, rectHeight6], 'FaceColor', 'y', 'EdgeColor', 'none');
523 for i = 1:N
524     hold on
525     set(l2_plot, 'XData', [p_O2(1), p_B(i,1)], 'YData', [p_O2(2), p_B(i,2)]);
526     set(l3_plot, 'XData', [p_O3(1), p_C(i,1)], 'YData', [p_O3(2), p_C(i,2)]);
527     set(l5_plot, 'XData', [p_C(i,1), p_D(i,1)], 'YData', [p_C(i,2), p_D(i,2)]);
528     set(rect_plot, 'Position', ...
529         [-rectWidth6/2 + p_D(i,1), -rectHeight6/2 + p_D(i,2), rectWidth6, rectHeight6]);
530
531     % Pause for animation effect
532     hold off
533     pause(0.01)
534 end
```

B Original Appendix Output Figures

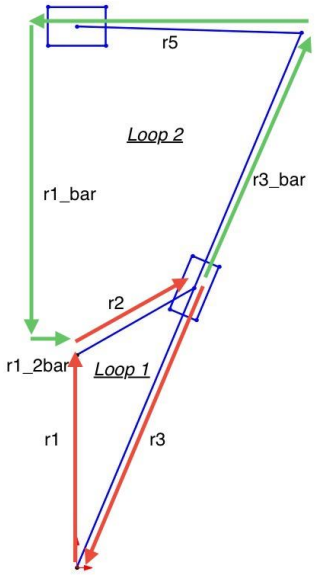


Figure 26: Original MATLAB appendix output 60.

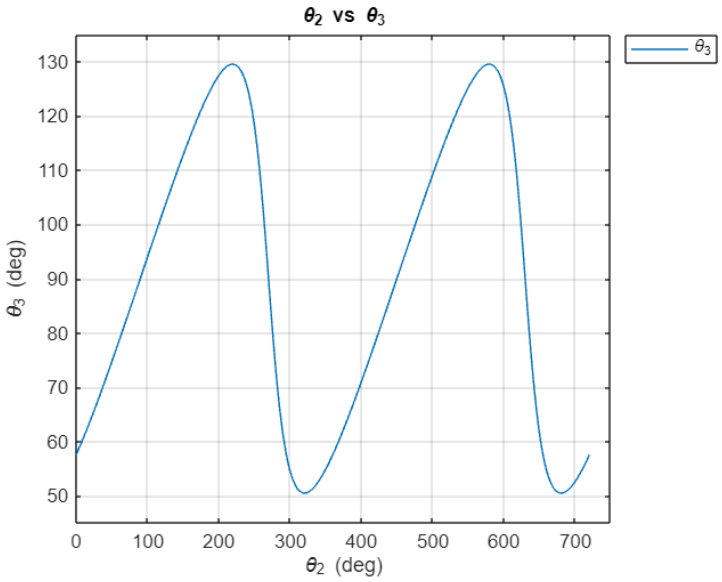


Figure 27: Original MATLAB appendix output 61.

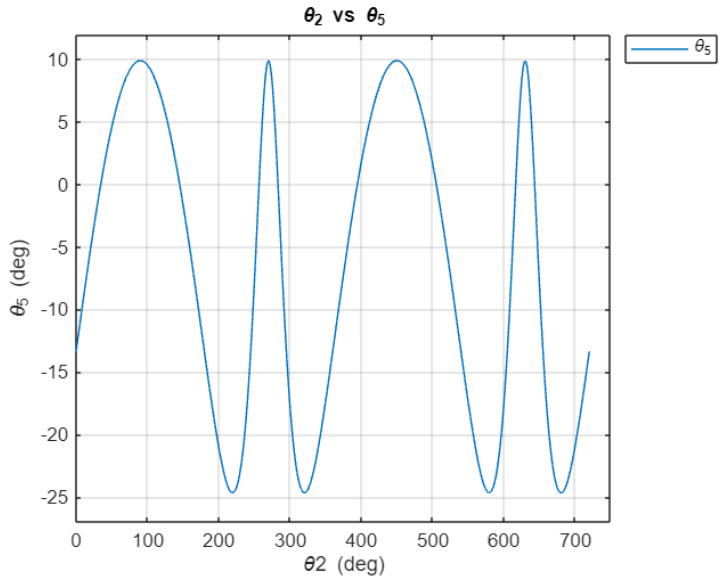


Figure 28: Original MATLAB appendix output 62.

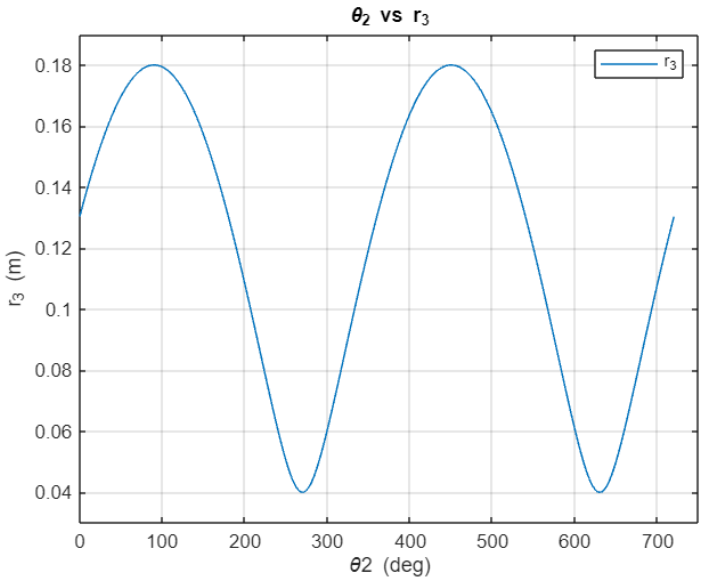


Figure 29: Original MATLAB appendix output 63.

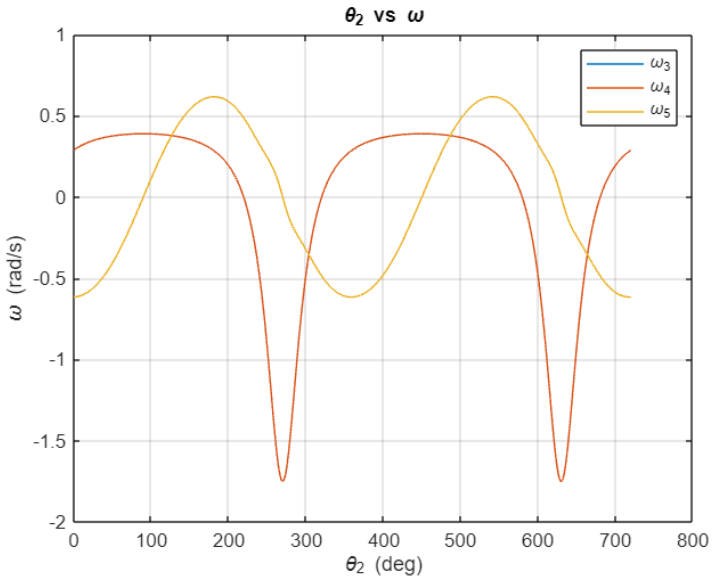


Figure 30: Original MATLAB appendix output 64.

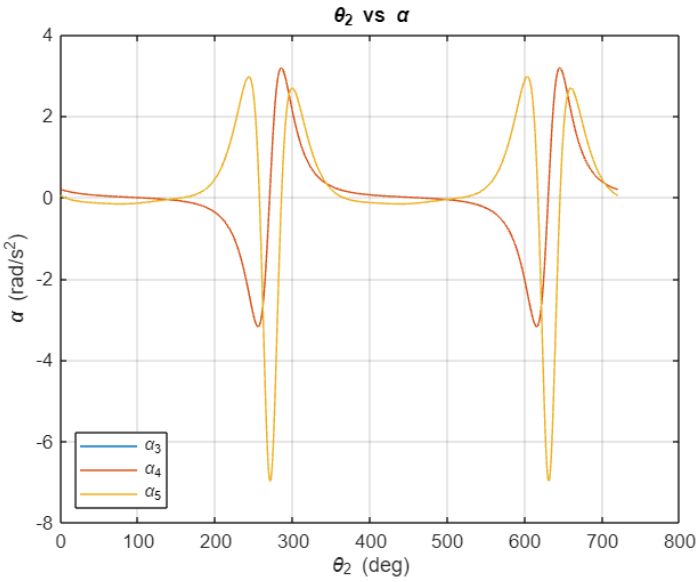


Figure 31: Original MATLAB appendix output 65.

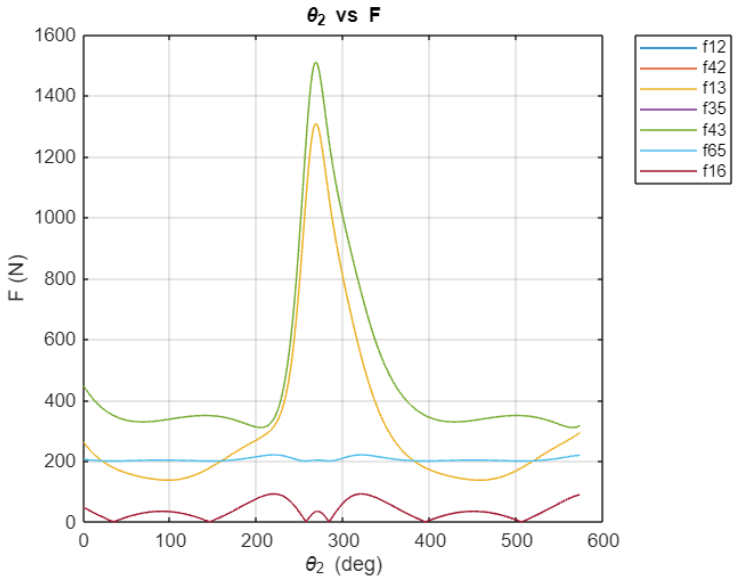


Figure 32: Original MATLAB appendix output 66.

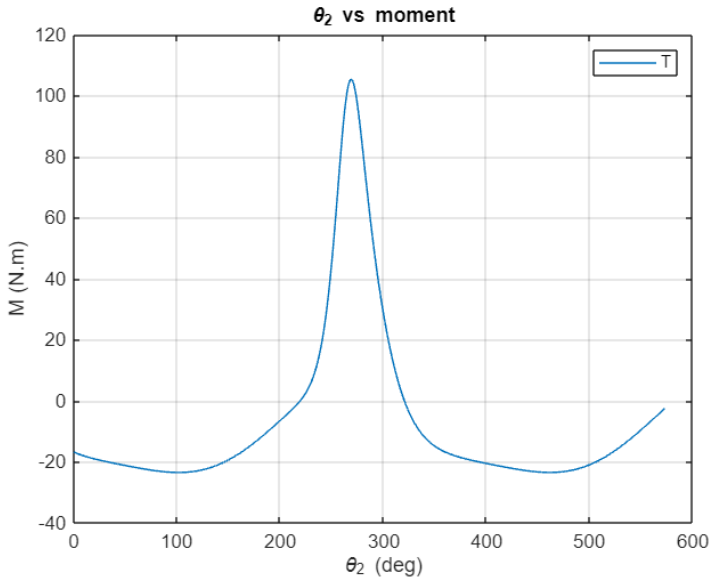


Figure 33: Original MATLAB appendix output 67.

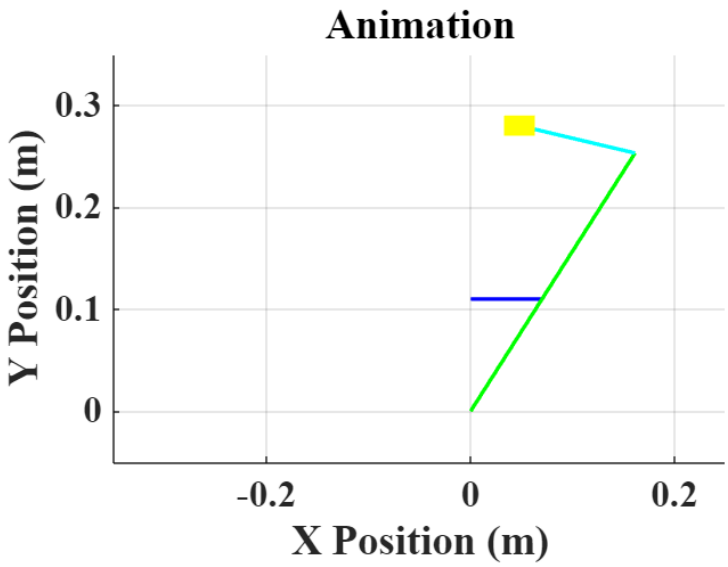


Figure 34: Original MATLAB appendix output 68.

C Editorial Review and Reproducibility

The original study provides software comparisons, not experimental validation or a quantified optimization objective. The observed member-5 angular-velocity mismatch remains unresolved. Similar-looking acceleration curves do not establish that the velocity equation is correct.

The MATLAB source merits review before reuse. In the second-loop velocity right-hand side, several displacement and derivative terms appear inside the sine/cosine arguments instead of being added as separate terms. In the first-loop acceleration expression, a term uses r3p where the expected radial centripetal term warrants checking against r3. The dynamic member-5 force equations use aG4x and aG4y rather than the separately calculated member-5 accelerations. These are inspection findings, not fixes that have been validated by a rerun.

The source assigns a density of 1 kg/m^3 and uses correspondingly small masses and inertias. These are modeling assumptions, not established material properties for the CAD parts. The statement that slider 6 is the heaviest member should be checked against the supplied mass list. Gravity, the 200 N load, coordinate signs, and mass units must be consistent between programs. Some report equations contain inconsistent subscripts and signs. The fourteen-equation matrix is retained as a source transcription; do not treat it as a verified implementation independent of the code.

The MATLAB study uses 2000 points over $0 \leq \theta_2 \leq 4\pi$, with input speed 1 rad/s and zero angular acceleration. The nominal two-cycle kinematic interval is therefore about 12.57 seconds. The dynamic subsection uses a shorter interval and a hardcoded plotting index; distinguish angle axes from time histories when comparing figures. The source's Adams description of a degree-based time increment is not a confirmed identical speed setting.

The original MATLAB Live Script and workspace, Adams binary models, SAM project, SolidWorks assembly and referenced parts, and available smaller videos are included. The Excel cam worksheet, coordinate CSV, and a separate curve-fit export are not supplied. Their screenshots are retained, but missing numerical source data have not been reconstructed. MATLAB requires Symbolic Math Toolbox for the symbolic expressions and matrix generation. A compatible release is needed for local functions placed amid script sections; the native Live Script is the preferred starting point.

The extracted text version preserves code blocks and turns Live Script prose headings into comments. It does not reproduce stored Live Editor output or embedded illustrations and is not claimed to be a tested standalone replacement. The report listing is a second historical source and may differ from the saved Live Script. Original files have not been altered.

Adams native models are compressed together in `adams/native_models.zip` to keep each browser-uploaded file within GitHub's 25 MiB limit. Extract that archive before opening the models. The approximately 62.4 MiB MATLAB video is omitted from this repository package and remains in the original RAR archive; it is not required to build the report. All other supplied videos are included. No native CAD rebuild, multibody rerun, or hardware experiment was performed during preparation.